An oracle blocks advice-free heuristic derandomization, with a conditional polynomial-space implementation

Shuichi Hirahara, Zhenjian Lu, Hanlin Ren

doi:10.4230/LIPIcs.CCC.2023.6; CC BY 4.0

Editorial note. Original complete official proofs of11 and12 retained as a single major oracle-barrier result, preserving all original hypotheses and literal mathematical expressions. Earlier complete construction10 is context for author references; no supporting lemma or direct corollary is separately counted. Files unique to this task added; frozenCC030 dependencies unchanged.

Allowed standard prerequisites (editorial).

- Approximate counting in the polynomial hierarchy, padding and search-to-decision reductions. Exact original Theorems23/24 retained, including author counting sketch; standard padding under the unchanged SAT assumption used as in author proof

Definitions and notations (original Sections 1.1, 2.1). For a complexity class $\mathfrak{C}$, a complexity-theoretic statement is $\mathfrak{C}$-relativizing if it is true relative to every oracle $\mathcal{O} \in \mathfrak{C}$.

Definition 20. A language L is in EXPH if there is a constant k, an exponential bound $e(n) = 2^{\mathrm{poly}(n)}$, and a $\mathrm{poly}(e(n))$-time algorithm M such that, for every input $x \in \{0,1\}^*$,

$$x \in L \iff \exists y_1 \forall y_2 \exists y_3 \forall y_4 \dots \exists y_k,\ M(x, y_1, \dots, y_k) = 1,$$

where $y_i \in \{0,1\}^{e(n)}$ for each $i \in [k]$. Equivalently, a language is in EXPH if it can be decided in exponential $(2^{\mathrm{poly}(n)})$ time with access to a PH oracle.

Pseudodeterministic algorithms. A pseudodeterministic algorithm is a probabilistic algorithm that returns a fixed output with high probability on any given input. We will also consider pseudodeterministic oracle (or advice-taking) algorithms. Such an algorithm will maintain its pseudodeterministic behavior (i.e., outputting a fixed answer with high probability for any input) when a correct oracle (or advice) is given.

Dense property (original Section 1.3.2). A property $Q \subseteq \{0,1\}^*$ is dense if for every input length $n \in \mathbb{N}$, we have that $|Q \cap \{0,1\}^n| \geq 2^n / \mathrm{poly}(n)$.

Theorem 23 [68]. For every oracle $\mathcal{O}$, $\mathsf{BPP}^{\mathcal{O}} \subseteq (\Sigma_2^p)^{\mathcal{O}}$.

Theorem24 (Approximate Counting in Quasi-Linear-Time Hierarchy). There is a $\Sigma_5\mathsf{TIME}[n \cdot \mathrm{polylog}(n)]$ machine M such that on input (φ, K), accepts if φ has at least K satisfying assignments, and rejects if φ has at most $K/2$ satisfying assignments.

Proof Sketch. Using universal hashing [69], we can show that there is a $\mathsf{BPTIME}[n \cdot \mathrm{polylog}(n)]^{\mathsf{SAT}}$ machine M' that approximates the number of satisfying assignments of φ'. Let H be a family of pairwise-independent hash functions whose output range is $\{0,1\}^{\lceil \log K \rceil}$, that is computable in near-linear time. (See, e.g., [72, Problem3.3,(2)] for an example based on Toeplitz matrices.) The machine M' randomly samples $h \sim H$ and $r \sim \{0,1\}^{\lceil \log K \rceil}$, and uses the SAT oracle to decide whether there is a satisfying assignment x of φ' such that $h(x) = r$. Finally, the lemma follows from the fact that for every oracle $\mathcal{O}$, $\mathsf{BPTIME}^{\mathcal{O}}[t] \subseteq \Sigma_3\mathsf{TIME}[t \cdot \mathrm{polylog}(t)]^{\mathcal{O}}$ [74]. □

Theorem 10. There is an oracle $\mathcal{O} \in \mathsf{EXPH}$ such that

$$\mathsf{RP}^{\mathcal{O}} \not\subseteq \text{i.o.}\,\mathsf{DTIME}^{\mathcal{O}}[2^n] \qquad \text{and} \qquad \mathsf{UP}^{\mathcal{O}} \not\subseteq \mathsf{BPTIME}^{\mathcal{O}}[2^n].$$

Proof of Theorem 10. Our oracle world will consist of two oracles P and Q (it is easy to combine them into a single oracle). The oracle P creates a hard problem in RP against deterministic algorithms, and the oracle Q creates a hard problem in UP against probabilistic algorithms. In particular, every input to the oracle P will be of the form (x, r) where $|r| = 100|x|$, and let

$$L_P := \{x : \exists r \in \{0,1\}^{100|x|}, P(x,r) = 1\}.$$

We will guarantee that for every x, $\Pr_r[P(x,r) = 1]$ is either 0 or greater than $1/2$, thus $L_P \in \mathsf{RP}$. We also define

$$L_Q := \{1^n : \exists r \in \{0,1\}^{100n}, Q(r) = 1\}.$$

We will guarantee that for every input length $100n$, $|Q \cap \{0,1\}^{100n}| \leq 1$, thus $L_Q \in \mathsf{UP}$. Let $M_1, M_2, \ldots$ be an enumeration of Turing machines running in deterministic 2^n time, and $N_1, N_2, \ldots$ be an enumeration of probabilistic Turing machines running in 2^n time (that does not necessarily satisfy the BPP promise).[1] We want every M_i to fail to solve L_P on all but finitely many input lengths, and every N_i to fail to solve L_Q on infinitely many input lengths. In particular, let n_1 be a large enough constant, and $n_i = 2^{500n_{i-1}}$ for each $i \geq 2$. Assuming that every machine appears in the sequence $\{N_i\}$ infinitely often, we want that for each $i \in \mathbb{N}$, N_i fails to solve L_Q on input 1^{n_i}. We also need the following terminologies. The n-th slice of P is the set of inputs of the form (x, r) where $|x| = n$ and $|r| = 100n$. The ℓ-th slice of Q is simply the set of inputs with length ℓ. During our construction, there will be some entries $P(x,r)$ and $Q(r)$ that are fixed, meaning that their values will never change in the subsequent construction. There will also be some inputs x that are frozen, meaning that for every $r \in \{0,1\}^{100|x|}$, the values of $P(x,r)$ are fixed and will never change in

[1] In this paper, whenever we enumerate a list of Turing machines, we assume every machine occurs in the list infinitely many times.

the subsequent construction. If an entry $P(x, r)$ or $Q(r)$ is not fixed and, in the case of $P(x, r)$, x is not frozen, then we say this entry is free. Our construction proceeds in stages. During stage n, we will fix the entire n-th slices of P and Q; we will possibly also fix or freeze some other entries. It will be guaranteed that before the n-th stage:

1. the number of frozen strings beyond the n-th slice (including the n-th slice) is at most $2^{n/2}$;
2. the number of fixed strings (except the frozen ones) beyond the n-th slice (including the n-th slice) is at most 2^{5n}; and
3. each fixed or frozen entry beyond the n-th slice (including the n-th slice) is set to be 0.

Diagonalization against M_i. Let $x_1, x_2, \ldots, x_n$ be n inputs of length n that are not frozen. Since there are at most $2^{n/2}$ frozen inputs of length n, it is always possible to select n unfrozen inputs. For each $i \in [n]$, we simulate the machine M_i on input x_i. Whenever M_i asks a query $P(x, r)$ or $Q(r)$, if this query is not free, then we return the corresponding value fixed before; if it is free, then we return 0 as the answer and also fix this query to be 0. Then M_i will output an answer $\mathsf{ans}_i \in \{0, 1\}$. Note that there are at most $2^{5n} + n \cdot 2^n \ll 2^{100n}/3$ entries in P that are fixed, thus the vast majority of the entries $P(x_i, r)$ are free. We fix all these entries $P(x_i, r)$ to be $(1 - \mathsf{ans}_i)$. It is easy to see that M_i fails to compute L_P on input x_i.

Diagonalization against N_i. If $n = 10n_i$ for some $i \in \mathbb{N}$, then we also need to fix the $100n_i$-th slice of Q so that N_i fails to compute L_Q on input 1^{n_i}. (In stage n, this happens after the above diagonalization against M_i.) We simulate N_i on input 1^{n_i}. Note that N_i is a probabilistic machine running in time 2^{n_i}, thus it has $B := 2^{2^{n_i}}$ computational branches, where each branch has $1/B$ probability mass. On each branch, whenever N_i asks a query $P(x, r)$ or $Q(r)$, if this query is not free, then we return the corresponding value fixed before; if it is free, then we return 0 as the answer to the query. Note that we cannot fix the query as there are potentially as many as B queries to fix. Instead, after simulating all the computational branches, we pick out the "heavy" queries that occur in a lot of branches and fix them:

- A string x is heavy if at least $\frac{1}{10^4|x|^3}$ fraction of branches queried some free entry of the form $P(x, r)$.
- An entry $Q(r)$ is heavy if at least $\frac{1}{10^4}$ fraction of branches queried $Q(r)$.

We freeze every heavy x by setting every unfixed entry $P(x, r)$ to be 0. We also fix every heavy $Q(r)$ to be 0. Note that each computational branch of N_i only makes at most 2^{n_i} queries. Therefore, for each input length m where $n \le m \le 2^{n_i}$, the number of heavy strings $x \in \{0, 1\}^m$ is at most $10^4 2^{n_i} m^3$. The number of entries $Q(r)$ fixed before this stage is at most 2^{5n}, and the number of heavy entries $Q(r)$ is at most $10^4 2^{n_i}$, therefore the total number of fixed entries in Q after this stage is at most $2^{6n} \ll 2^{100n_i}$. After simulating every computational branch of N_i, let p be the probability that N_i outputs 1 on input

1^n. If $p < 0.5$, then we pick a free $r \in \{0,1\}^{100n_i}$ and set $Q(r) = 1$; since the number of fixed entries $Q(r)$ is $\ll 2^{100n_i}$, such r always exists. If $p \geq 0.5$, then we do nothing. Now we have that N_i solves L_Q on input 1^{n_i} with probability less than $2/3$ (actually, at most $1/2$). Is this always the case in the future? Consider how the future changes to the oracles might affect the behavior of N_i. Let m be an input length where $n < m \leq 2^n$. During stage m (which has not happened yet), we will pick m inputs $x_1, x_2, \ldots, x_m \in \{0,1\}^m$, then some entries of the form $P(x_i, r)$ will be set to be 1. Also note that the probability mass of computational branches that each x_i affects is at most $\frac{1}{10^4 m^3}$. Therefore, the total probability mass of computational branches affected is at most

$$\sum_{m \in \mathbb{N}} \frac{m}{10^4 m^3} < \frac{1}{10^3}.$$

Note that we also set at most one entry $Q(r)$ to be 1, where $|r| = 100n_i$. (The next time we diagonalize against N_{i+1} happens in stage $n_{i+1} \gg 2^n$.) Since this entry $Q(r)$ is not heavy, the probability mass of computational branches that it affects is at most $\frac{1}{10^4}$. It follows that only an $\frac{11}{10^4}$ fraction of computational branches of N_i will be affected in the future. That is, let p' be the probability that N_i outputs 1 on input 1^{n_i} at the end of our construction (i.e., after stage $\gg 2^n$), then $|p' - p| < \frac{11}{10^4}$. Therefore, it cannot be the case that

$$\Pr[N_i(1^{n_i}) = L_Q(1^{n_i})] \geq 2/3.$$

Clean-up. At the end of stage n, we fix every unfixed input $P(x, r)$ in the n-th slice to be 0. If $n = 10n_i$ then we also fix every unfixed input $Q(r)$ in the $10n$-th slice to be 0. The number of frozen strings is at most $10^4 2^{n/10} n^3 \leq 2^{n/2}$ (note that at most one N_i contributes to the frozen strings since the input lengths $\{n_i\}$ are very far apart). The number of fixed entries above the $(n+1)$-st slice is at most

$$2^{5n} + n \cdot 2^n + \sum_{m=n}^{2^{n/10}} 10^4 2^{n/10} m^3 < 2^{5(n+1)}.$$

The complexity of $\mathcal{O}$. We show that the above oracles P and Q are computable in EXPH. It suffices to present a deterministic algorithm with access to a Σ_3^p oracle that on input 1^n, outputs the truth tables of the n-th slices of P and Q in $2^{O(n)}$ time. We simulate each stage $n' \leq n$. Before stage n', we maintain the set of fixed entries $P(x, r), Q(r)$, as well as the set of frozen strings x. There are at most $2^{O(n')}$ many such strings. In what follows, we use $\langle P, Q \rangle$ to denote (the length-$2^{O(n')}$ encoding of) the list of fixed entries and frozen strings. It is easy to diagonalize against each M_i: we simply choose the inputs $x_1, x_2, \ldots, x_{n'}$, simulate each M_i on input x_i, and fix the oracles accordingly. This takes $2^{O(n')}$ time. To simulate N_i, we need to define the following language capturing the weight of queries. Consider simulating the probabilistic machine N_i on the current oracle $\langle P, Q \rangle$. Whenever N_i makes a query, if this query is not free, then we return the answer of the query; otherwise we return 0. The following two claims are immediate corollaries of Theorem 23:

Claim 42. There is a language $\mathsf{Heavy\text{-}String}(\langle P, Q \rangle, N_i, x)$ computable in Σ_2^p such that:

- if at least $\frac{1.1}{10^4|x|^3}$ fraction of computational branches of N_i queried some free entry of the form $P(x,-)$, then $\mathsf{Heavy\text{-}String}(\langle P,Q\rangle, N_i, x) = 1$;
- if at most $\frac{0.9}{10^4|x|^3}$ fraction of computational branches of N_i queried some free entry of the form $P(x,-)$, then $\mathsf{Heavy\text{-}String}(\langle P,Q\rangle, N_i, x) = 0$.

Claim 43. There is a language $\mathsf{Heavy\text{-}Entry}(\langle P,Q\rangle, N_i, r)$ computable in Σ_2^p such that:

- if at least $\frac{1.1}{10^4}$ fraction of branches of N_i queried $Q(r)$, then $\mathsf{Heavy\text{-}Entry}(\langle P,Q\rangle, N_i, r) = 1$;
- if at most $\frac{0.9}{10^4}$ fraction of branches of N_i queried $Q(r)$, then $\mathsf{Heavy\text{-}Entry}(\langle P,Q\rangle, N_i, r) = 0$.

(Note that both $\mathsf{Heavy\text{-}String}$ and $\mathsf{Heavy\text{-}Entry}$ accepts inputs of length $2^{O(n')}$.) With the aid of these two languages, it is possible to simulate stage n' now. We say a string x is heavy if $\mathsf{Heavy\text{-}String}(\langle P,Q\rangle, N_i, x) = 1$, and an entry $Q(r)$ is heavy if $\mathsf{Heavy\text{-}Entry}(\langle P,Q\rangle, N_i, r) = 1$. Since there are at most $2^{O(n')}$ heavy strings, we can find all of them in $2^{O(n')}$ time with access to an $\mathsf{NP}^{\mathsf{Heavy\text{-}String}}$ oracle. Similarly, since there are $2^{O(n')}$ heavy entries, we can find all of them in $2^{O(n')}$ time with access to an $\mathsf{NP}^{\mathsf{Heavy\text{-}Entry}}$ oracle. We freeze these heavy strings and fix these heavy entries accordingly. Finally, we estimate the probability that N_i outputs 1 (given current versions of P and Q) in Σ_2^p, within additive error 0.1. If this estimation is less than $1/2$, then we pick a free $r \in \{0,1\}^{10n'}$ and set $Q(r) = 1$; otherwise we do nothing. At the end of stage n', we fix every unfixed entry in the n'-th slice of $P(x,r)$ to be 0. If $n = 10n_i$ then we also fix every unfixed input $Q(r)$ in the $10n$-th slice to be 0. This takes $2^{O(n')}$ time. It is easy to see that simulating the n stages takes $2^{O(n)}$ time with access to a Σ_3^p oracle. □

Theorem11. There is an oracle $\mathcal{O}$ such that

$$\mathsf{BPP}^{\mathcal{O}} \not\subseteq \text{heur-}\mathsf{DTIME}^{\mathcal{O}}[2^n] \qquad \text{and} \qquad \mathsf{UP}^{\mathcal{O}} \not\subseteq \text{i.o.}\,\mathsf{BPTIME}^{\mathcal{O}}[2^n].$$

Theorem12. Suppose that $\mathsf{SAT} \in \mathsf{DTIME}[n \cdot \text{polylog}(n)] \cap \mathsf{NC}$. Then there is an oracle $\mathcal{O}$ satisfying the conclusions of Theorem11 that can be computed in polynomial space.

Proof of Theorem11. We use a similar construction as in Theorem10. The difference is that now we want a problem in BPP that is infinitely-often hard, and a problem in UP that is almost-everywhere hard, therefore we need to define the oracles differently. In particular, the oracle P only receives one input r, and the oracle Q receives two inputs (x,r), where $|r| = 10|x|$. The hard language in BPP is:

$$L_P = \{1^n : \Pr_{r\sim\{0,1\}^{n^4}}[P(r) = 1] \geq 1/2\}.$$

It is guaranteed that for every n, the fraction of length-n^4 strings that are in P is either at most $1/3$ or at least $2/3$, therefore $L_P \in \mathsf{BPP}$. On the other hand, the hard language in UP is:

$$L_Q = \{x : \exists r \in \{0,1\}^{10|x|}, Q(x,r) = 1\}.$$

It is guaranteed that for every input x, there is at most one string $r \in \{0,1\}^{10|x|}$ such that $Q(x,r) = 1$, therefore $L_Q \in \mathsf{UP}$. We will construct the oracles P and Q such that $L_P \notin \mathsf{DTIME}[2^n]$ and $L_Q \notin \text{i.o.}\,\mathsf{BPTIME}[2^n]$. Note that L_P is a unary language, therefore we also have $\mathsf{BPP} \not\subseteq \text{heur-}\mathsf{DTIME}[2^n]$. We use the same terminologies as before: $M_1, M_2, \ldots$ is an enumeration of deterministic Turing machines running in 2^n time, and $N_1, N_2, \ldots$ is an enumeration of probabilistic Turing machines running in 2^n time. The ℓ-th slice of P is the set of inputs with length ℓ, and the n-th slice of Q is the set of inputs of the form (x,r) where $|x| = n$ and $|r| = 10n$. An entry is fixed if its value will never change in the subsequent construction. We choose n_1 to be a large enough constant and define a sequence $\{n_i\}$ where $n_i = 2^{10n_{i-1}}$ for each $i \geq 2$. We want every N_i to fail to solve L_Q on all but finitely many input lengths, and every M_i to fail to solve L_P on input 1^{n_i}. Our construction proceeds in stages. We guarantee that:

- For every input length n that is not in the sequence $\{n_i\}$, the n-th slice of P is identically 0.
- For every input length n, there are at most $2n$ entries in the entire n-th slice of Q that returns 1.
- Consider only the oracle Q. Before the n-th stage, the number of fixed entries beyond the n-th slice of Q (including the n-th one) is at most 2^{6n}, and all of them are fixed to be 0.

We start from the $n^\star$-th stage for some large enough constant $n^\star$. For each $n \geq n^\star$, in the n-th stage, we diagonalize against the machines N_i and M_i, and fix the n-th slice of Q as well as the n^4-th slice of P (when n equals some n_i). Details follow.

Diagonalization against N_i. Let n' be the smallest integer in the sequence $\{n_i\}$ such that $n' \geq n$, and i' be the index such that $n' = n_{i'}$. The $(n')^4$-th slice of P may contain either few or most strings, and we do not know which is the case yet. Therefore, we will diagonalize each N_i twice, first assuming the n'-th slice of P is nearly empty and then assuming the n'-th slice of P is nearly full. For this reason, we will need $2n$ distinct strings $x_{i,b} \in \{0,1\}^n$, one for each pair of $i \in [n]$ and $b \in \{0,1\}$. Note that at the beginning of the n-th stage, there are at most $2^{6n} \ll 2^{10n}$ strings fixed in the n-th slice of Q, therefore we can always choose $2n$ strings $x_{i,b}$ where no entry of the form $Q(x_{i,b}, r)$ is fixed. For each $i \in [n]$ and $b \in \{0,1\}$, we simulate $N_i(x_{i,b})$, assuming that most entries in the n'-th slice of P returns b. Note that N_i is a probabilistic machine running in time 2^n, thus it has $B := 2^{2^n}$ computational branches, where each branch has $1/B$ probability mass. On each branch, whenever N_i asks a query $P(r)$ or $Q(x,r)$, if this query is already fixed, then we return the corresponding value fixed before; otherwise:

- Suppose the query is $P(r)$. If $|r|$ is in the sequence $\{n_i^4\}$, then since $P(r)$ is not fixed and $|r| \leq 2^n$, we know that $|r| = (n')^4$ and we return b as the answer. Otherwise ($|r| \neq (n')^4$) we return 0.
- Suppose the query is $Q(x,r)$, then we return 0 as the answer.

Again, we can only afford to fix the heavy entries, defined as follows. Let K be the number of machine-input pairs that we still need to simulate in the future before fixing the $(n')^4$-th slice of P; note that this includes both the current $N_i(x_{i,b})$ and the final $M_{i'}(1^{n'})$. That is:

$$K = 2(n-i) + (2-b) + \sum_{j=n+1}^{n'} (2j) + 1.$$

- An entry $P(r)$ is heavy if $|r| = (n')^4$ and at least $2^{-(n'+4)K}$ fraction of branches queried $P(r)$.
- An entry $Q(x, r)$ is heavy if at least $\frac{1}{10^4|x|^3}$ fraction of branches queried $Q(x, r)$.

We fix every heavy entry according to the way we answer this entry before. That is, every heavy entry $P(r)$ is fixed to be b and every heavy entry $Q(x, r)$ is fixed to be 0. The number of heavy entries in P is at most $2^{(n'+4)K} \cdot 2^n$; for each input length $m \geq n$, the number of heavy entries in the m-th slice of Q is at most $2^n \cdot 10^4 \cdot m^3$. Let p be the probability (over the B computational branches) that $N_i(x_{i,b})$ outputs 1. If $p < 1/2$ then we find a string r such that $Q(x_{i,b}, r)$ is unfixed yet and set $Q(x_{i,b}, r) = 1$, making $x_{i,b} \in L_Q$. Such a string r exists since we have fixed strictly less than 2^{10n} strings in the n-th slice of Q. If $p \geq 1/2$ then we do nothing. Assuming that most entries in the $(n')^4$-th slice of P indeed returns b, at this point the probability that N_i outputs the correct answer on input $x_{i,b}$ is at most $1/2$. However, the oracles P and Q might change in the future, thus the behavior of N_i might also change. What fraction of computational branches might be affected in the future?

- Consider the $K-1$ machine-input pairs that we will simulate in the future before the $(n')^4$-th slice of P is completely fixed. The number of heavy entries in the $(n')^4$-th slice of P that these machines fix is at most

  $$\sum_{k=1}^{K-1} 2^{(n'+4)k} \cdot 2^{n'} \leq 2 \cdot 2^{(n'+4)(K-1)} \cdot 2^{n'}.$$

  Therefore, the probability mass of computational branches of $N_i(x_{i,b})$ influenced by the future changes to the $(n')^4$-th slice of P is at most

  $$2 \cdot 2^{(n'+4)(K-1)} \cdot 2^{n'} \cdot 2^{-(n'+4)K} \leq 1/8.$$

- For each $m \leq 2^n$, there are at most $2m$ entries in the m-th slice of Q that we will fix to be 1. The probability mass of computational branches of $N_i(x_{i,b})$ influenced by these entries is at most

  $$\sum_{m=n}^{2^n} \frac{2m}{10^4 m^3} < \frac{1}{10^3}.$$

It follows that the future modifications to the oracles P and Q will only affect the accept probability of $N_i(x_{i,b})$ by $\frac{1}{8} + \frac{1}{10^3} < \frac{1}{6}$. Therefore, it cannot be the case that

$$\Pr[N_i(x_{i,b}) = L_Q(x_{i,b})] \geq 2/3.$$

Diagonalization against M_i. If $n = n_i$ for some $i \in \mathbb{N}$, then we want that M_i does not compute L_P on input 1^n correctly. We simulate M_i on the input 1^n. When M_i asks a query $P(r)$ or $Q(x, r)$, if this query is already fixed, then we return the corresponding value fixed before; otherwise we return 0 and fix this queried entry. After this simulation, we argue that only a small fraction of entries in the n^4-th slice of P are fixed. First, the simulation of $M_i(1^n)$ fixes at most 2^n entries. Second, there are at most $K = \sum_{j=1}^{n}(2j) = O(n^2)$ machine-input pairs $N_i(x_{i,b})$ simulated so far, and the k-th such machine fixes at most $2^{(n+4)k} \cdot 2^n$ entries. Therefore, the number of entries fixed by some previous $N_i(x_{i,b})$ is at most

$$\sum_{k=1}^{K} 2^{(n+4)k} \cdot 2^n \leq 2^{O(n^3)}.$$

It follows that all but $2^{O(n^3)}$ entries in the n^4-th slice of P are not fixed yet. Let $b \in \{0, 1\}$ be the output bit of $M_i(1^n)$, then we set $L_P(1^n) = 1 - b$ by fixing every unfixed entry in the n^4-th slice of P to be $1 - b$. We have that on input 1^n, L_P satisfies the BPP promise, and M_i fails to solve L_P.

Clear-up. At the end of stage n, we fix every unfixed input $Q(x, r)$ on the n-th slice to be 0. It is easy to see that there are at most $2n$ entries in the n-th slice of $Q(x, r)$ that returns 1. During the n-th stage, the number of entries we fixed beyond the $(n + 1)$-st slice of Q is at most

$$2^n + \sum_{m=n+1}^{2^n} 2^n \cdot 10^4 \cdot m^3 \leq 2^{5n}.$$

Thus the total number of entries we fixed beyond the $(n + 1)$-st slice of Q is at most $2^{6n} + 2^{5n} < 2^{6(n+1)}$. □

Unfortunately, we do not know how to compute the above oracles in EXPH. The reason is that when we diagonalize N_i on input length n, the number of heavy entries $P(r)$ we need to fix is $2^{(n'+4)K} \cdot 2^n = 2^{\mathrm{poly}(n')}$. Since n' might be exponentially large compared to n, this upper bound might be doubly exponential. Nevertheless, we show that under the assumption that

$$\mathsf{SAT} \in \mathsf{DTIME}[n \cdot \mathrm{polylog}(n)] \cap \mathsf{NC}, \tag{*}$$

the oracles P, Q in Theorem11 can be computed in PSPACE. We sketch some intuition before presenting the full proof of Theorem12. The bottleneck of putting $\mathcal{O}$ into EXPH is that we do not have enough space to store all the heavy entries $P(r)$ fixed by a probabilistic machine N_i, as there might be doubly-exponentially many of them. Therefore, whenever we simulate a machine N_i and it asks a query $P(r)$, we have to compute from scratch whether $P(r)$ was fixed by a previous machine. If we have an oracle Heavy that given r, decides whether $P(r)$ was already fixed, then we can simulate N_i in exponential time with a constant number of alternations (using Theorem24). However, there are $\mathrm{poly}(n)$ machines N_j simulated before N_i, and each time we invoke Heavy, we need to simulate these machines to see if any of them has already fixed $P(r)$. It follows that the simulation of N_i actually requires $\mathrm{poly}(n)$ alternations and $2^{\mathrm{poly}(n)}$ time. Still, under a strong enough assumption such as (*), we can simulate these $\mathrm{poly}(n)$ alternations in $2^{\mathrm{poly}(n)}$ time.

Proof of Theorem12. Consider the construction in Theorem11, with the only difference that we do not calculate the precise probabilities; instead, we compute their approximations in PH. Since $\mathsf{SAT} \in \mathsf{NC}$, we have $\mathsf{EXP} = \mathsf{PSPACE}$ by padding, thus it suffices to construct the oracles in $2^{\mathrm{poly}(n)}$ time. The most involved part of this proof is to compute the following function $\mathsf{Heavy}(n, \langle P, Q\rangle, i, b, x_{i,b}, r)$, which indicates whether $P(r)$ is a heavy query of the probabilistic machine N_i on input $x_{i,b}$. Here, the input of Heavy consists of the description of oracles P and Q, an integer $i \le n$, an input $x_{i,b} \in \{0,1\}^n$, and a query $r \in \{0,1\}^{(n_{i'})^4}$ (where $n_{i'}$ is the smallest element in the sequence $\{n_i\}$ such that $n_{i'} \ge n$). We may assume $|r| \le 2^n$ as otherwise N_i could never query $P(r)$. The oracle description $\langle P, Q\rangle$ will contain the following information:

- We record a table of all entries in Q that are fixed; there are at most $\sum_{m \le n} 2m + 2^{6n} \le 2^{O(n)}$ such entries. For each entry, we also record a timestamp indicating when this entry is fixed.
- We record all entries of P up to the $n_{i'-1}$-th slice (but we do not include any information of the $n_{i'}$-th slice of P); the description length of P is also at most $2^{O(n)}$.

Note that the input length of Heavy is at most $2^{O(n)}$. Instead of requiring Heavy to decide exactly whether $P(r)$ is a heavy query, we only require a 2-approximation: if $P(r)$ is queried by $N_i(x_{i,b})$ w.p. at least $2^{-(n'+4)K}$ then Heavy returns 1, while if $P(r)$ is queried w.p. at most $2^{-(n'+4)K}/2$ then Heavy returns 0. Let k be the number of machine-input pairs we have simulated before $N_i(x_{i,b})$ (inclusive). That is,

$$k := \sum_{m<n} (2m) + 2i + b + 1 \le O(n^2).$$

We will show that:

Claim46. If $\mathsf{SAT} \in \mathsf{DTIME}[n \cdot \mathrm{polylog}(n)]$, then we can compute Heavy in deterministic $O(2^{k^2})$ time.

Proof. Consider simulating (a random computational branch of) $N_i(x_{i,b})$ while answering the oracle queries of N_i accordingly.[2] Note that we have not recorded the $n_{i'}$-th slice of P in our description $\langle P, Q\rangle$. As a consequence, whenever $N_i(x_{i,b})$ asks a query $P(r_q)$ where $|r_q| = n_{i'}$, we need to recursively call Heavy to decide whether this query was already fixed and which value it was fixed to. We enumerate every $n_{i'-1} < \tilde{n} \le n$, every machine N_j ($j \le \tilde{n}$), and every bit b'. Suppose that we feed the machine N_j with the input $x'_{j,b'} \in \{0,1\}^{n_{i'}}$.[3] Using the timestamps recorded in the table Q, we can recover the state of the oracle Q before the simulation of $N_j(x'_{j,b'})$; we call this oracle Q'. (The portion of oracle P up to the $n_{i'-1}$-th slice remains the same.) Then, we call $\mathsf{Heavy}(\tilde{n}, \langle P, Q'\rangle, j, b', x'_{j,b'}, r_q)$ to see if this query is a heavy query fixed by

[2] That is, whenever N_i asks a query $P(r)$ or $Q(x, r)$ that is fixed, we answer accordingly. Whenever N_i asks a query $P(r)$ that is not fixed, if $|r| = (n')^4$ then we return b, otherwise we return 0. Whenever N_i asks a query $Q(x, r)$ that is not fixed, we return 0.

[3] We assume that there is an easy and deterministic way of assigning the inputs $x_{j,b'}$ for each pair (j, b').

N'_j. If it is, then we return $P(r_q) := b'$; otherwise we search through the next machine. If the query $P(r)$ was not fixed before and was asked in this simulation. then we return 1; otherwise we return 0. The above argument implies that Heavy can be computed recursively in the following sense. There is a machine V_{Heavy} that gets $\mathsf{input} := (n, \langle P, Q\rangle, i, b, x_{i,b}, r)$ and some randomness $z \in \{0,1\}^{2^{O(n)}}$ (denoting a random computational branch of $N_i(x_{i,b})$), runs in $2^{O(n)}$ time with $\mathrm{poly}(n)$ invocations of the Heavy oracle, and outputs 0 or 1. Let $\varepsilon := 2^{-(n'+4)/K}$, we want that $\mathsf{Heavy}(\mathsf{input}) = 1$ if $\Pr_z[V_{\mathsf{Heavy}}(\mathsf{input}; z) = 1] \geq \varepsilon$, and $\mathsf{Heavy}(\mathsf{input}) = 0$ if $\Pr_z[V_{\mathsf{Heavy}}(\mathsf{input}; z) = 1] \leq \varepsilon/2$. We can use induction to show that Heavy can be computed in (deterministic) $O(2^{k^2})$ time. In particular, by the induction hypothesis, each call of the Heavy oracle made by V_{Heavy} on input can be computed in $O(2^{(k-1)^2})$ time. Therefore, V_{Heavy} runs in at most $2^{(k-1)^2} \cdot \mathrm{poly}(k)$ time. By Theorem24, there is a $\Sigma_5\mathsf{TIME}[2^{(k-1)^2} \cdot \mathrm{poly}(k)]$ time machine $\widehat{\mathsf{Heavy}}$ such that $\widehat{\mathsf{Heavy}}(\mathsf{input}) = 1$ if $\Pr_z[V_{\mathsf{Heavy}}(\mathsf{input}; z) = 1] \geq \varepsilon$ and $\widehat{\mathsf{Heavy}}(\mathsf{input}) = 0$ if $\Pr_z[V_{\mathsf{Heavy}}(\mathsf{input}; z) = 1] \leq \varepsilon/2$. By our hypothesis that $\mathsf{SAT} \in \mathsf{DTIME}[n \cdot \mathrm{polylog}(n)]$, there is a deterministic machine Heavy that runs in $2^{(k-1)^2} \cdot \mathrm{poly}(k) < 2^{k^2}$ time and decides the same language as $\widehat{\mathsf{Heavy}}$. □

Now we show that given an integer N, it is possible to print the N-th slice of P and Q in deterministic $2^{\mathrm{poly}(N)}$ time. We simulate the stages n for $n = 1, 2, \ldots$, where during each stage we need to diagonalize against probabilistic machines $N_1, N_2, \ldots$. If $n = n_i$ for some $i \in \mathbb{N}$, then we also need to diagonalize against M_i.

- For each N_i, we can list the set of its heavy queries of the form $Q(x, r)$ in deterministic $2^{\mathrm{poly}(n)}$ time with a $\mathsf{PH}^{\mathsf{Heavy}}$ oracle. Note that Heavy can be decided in time quasi-polynomial in its input length; also recall we assumed that $\mathsf{SAT} \in \mathsf{P}$. Thus we can enumerate the heavy queries of the form $Q(x, r)$ in deterministic $2^{\mathrm{poly}(n)}$ time (without additional oracles). We fix all these queries. (Note that we do not fix the heavy queries of the form $P(r)$; instead, we use the oracle Heavy to decide whether a query $P(r)$ is fixed.) Then we use the PH oracle to estimate the probability that $N_i(x_{i,b})$ outputs 1. If the probability is small then we pick some r such that $Q(x_{i,b}, r)$ is unfixed, and fix this entry to 1; otherwise we do nothing.
- Before we diagonalize against M_i, we spend $2^{\mathrm{poly}(n)}$ time to retrieve the list of fixed entries in the n^4-th slice of P from the oracle Heavy. Then we simulate M_i, and for every unfixed query, we fix it to be 0. Let $b \in \{0,1\}$ be the output of M_i, then we set every unfixed entry in the n^4-th slice of P to be $1 - b$.

It is easy to see that the above process runs in $2^{\mathrm{poly}(N)}$ time. □

References

[31] Shafi Goldwasser, Russell Impagliazzo, Toniann Pitassi, and Rahul Santhanam. On the pseudo-deterministic query complexity of NP search problems. CCC2021, LIPIcs200,36:1–36:22. doi:10.4230/LIPIcs.CCC.2021.36.

[33] Shuichi Hirahara. Non-black-box worst-case to average-case reductions within NP. FOCS2018,247–258. doi:10.1109/FOCS.2018.00032.

[34] Shuichi Hirahara. Non-disjoint promise problems from meta-computational view of pseudorandom generator constructions. CCC2020,LIPIcs169,20:1–20:47. doi:10.4230/LIPIcs.CCC.2020.20.

[68] Michael Sipser. A complexity theoretic approach to randomness. STOC1983,330–335. doi:10.1145/800061.808762.

[13] Harry Buhrman, Lance Fortnow, and Thomas Thierauf. Nonrelativizing separations. CCC1998,8–12. doi:10.1109/CCC.1998.694585.

[69] Larry J. Stockmeyer. The complexity of approximate counting (preliminary version). STOC1983,118–126. doi:10.1145/800061.808740.

[72] Salil P. Vadhan. Pseudorandomness. Foundations and Trends in Theoretical Computer Science,7(1–3):1–336,2012. doi:10.1561/0400000010.

[74] Emanuele Viola. On approximate majority and probabilistic time. CCC2007,155–168. doi:10.1109/CCC.2007.16.